

Arquitectura, flujo y ejecución del proyecto fibonacci_tmm

Emanuel Orozco Gallego

31 de agosto de 2026

Resumen

Manual de arquitectura del repositorio que reproduce Reyes-Gómez *et al.*, Phys. Rev. B **81**, 153101 (2010). Describe cómo está organizado el código, cómo circulan los datos desde un YAML hasta una figura, qué dependencias hacen falta y cómo instalar y ejecutar el proyecto de punta a punta. La física está en *Guía conceptual*; el algoritmo TMM, en *Solución numérica e implementación*.

Índice

1. Qué es este repositorio	3
2. Árbol del proyecto	3
3. Arquitectura de la biblioteca	3
3.1. Módulos y responsabilidades	3
3.2. Objetos que cruzan las fronteras	4
4. Capa de configuración	4
5. Capa de scripts	4
6. Flujo de una corrida	5
7. Tests	5
8. Dependencias	6
8.1. Tiempo de ejecución (Python)	6
8.2. Sistema, para los PDFs	6
8.3. Opcional	6
9. Cómo ejecutar el proyecto	6
9.1. 1. Entorno	6
9.2. 2. Tests (recomendado primero)	7
9.3. 3. Figuras	7
9.4. 4. Documentos PDF	7
9.5. 5. Todo de una vez	7
9.6. Cambiar un parámetro y volver a correr	7
9.7. Limpieza	7
10. Problemas frecuentes	7
11. Dónde está cada documento	8

1. Qué es este repositorio

Es una reimplementación en Python del método de matriz de transferencia (TMM) del Brief Report de 2010. No hay servidor web ni interfaz gráfica: el producto son *datos*, *figuras* y *PDFs*.

Tres capas, de dentro hacia afuera:

1. **Biblioteca** `src/fibonacci_tmm/`: física y numerica. No sabe de Matplotlib ni de números de figura.
2. **Scripts** `scripts/`: leen YAML, llaman a la biblioteca, dibujan y escriben archivos.
3. **Documentos** `docs/`: LaTeX que incrusta las figuras generadas.

Los parámetros científicos no viven en el código: viven en `configs/figure_ON.yaml`. Cambiar un plasma o un ángulo no exige editar Python.

2. Árbol del proyecto

<code>analisis_numerico/</code>	
<code>src/fibonacci_tmm/</code>	biblioteca (layout src)
<code>tests/</code>	pytest
<code>configs/</code>	YAML por figura + base.yaml
<code>scripts/</code>	reproduccion de Figs. 1-6
<code>figures/figure_ON/</code>	data/, output/, README.md
<code>figures/reproduced/</code>	copia de las imagenes finales
<code>docs/</code>	auditoria, paper, guias, este documento
<code>data/</code>	crudo / procesado / referencia (reserva)
<code>notebooks/</code>	reserva
<code>pyproject.toml</code>	paquete, pytest, pythonpath
<code>requirements.txt</code>	pip
<code>Makefile</code>	tests, figuras, PDFs

El entorno virtual `.venv/` y las cachés (`__pycache__`, `.pytest_cache`) no forman parte del código fuente. `.gitignore` también excluye auxiliares de LaTeX y el directorio `_paper_pages/` (recortes del PDF original).

3. Arquitectura de la biblioteca

El paquete usa el *src layout*: las importaciones son `from fibonacci_tmm...` y Setuptools busca paquetes en `src/` (`pyproject.toml`). Pytest añade `src` a `PYTHONPATH`. Los scripts hacen `sys.path.insert` a `src/` al arrancar.

3.1. Módulos y responsabilidades

Módulo	Depende de	Responsabilidad
<code>constants.py</code>	—	c , GHz, mm, tolerancias
<code>fibonacci.py</code>	constants	S_m , F_m , L_m
<code>materials.py</code>	—	aire y Drude $\varepsilon(\omega)$, $\mu(\omega)$
<code>electromagnetics.py</code>	constants	n , q , Q , χ , TE/TM
<code>params.py</code>	materials, electromagnetics	YAML $\rightarrow$ SI
<code>transfer_matrix.py</code>	electromagnetics, fibonacci, params	M , T_m , R_m
<code>dispersion.py</code>	transfer_matrix	barrido, bandas, kL_m/π
<code>plasmon_modes.py</code>	dispersion, fibonacci	recuento de subbandas
<code>bandwidth.py</code>	dispersion, plasmon_modes	$\nu_{\min}$, $\nu_{\max}$ vs θ
<code>plotting.py</code>	dispersion, matplotlib	estilo PRB y curvas
<code>provenance.py</code>	params, constants	metadatos JSON de cada corrida
<code>__init__.py</code>	los anteriores	API pública

La dependencia es acíclica y va de lo más físico a lo más aplicado: constantes → materiales y Fibonacci → TMM → barrido → modos y anchos → dibujo.

`plotting.py` es el único módulo de la biblioteca que importa Matplotlib. El núcleo numérico puede usarse sin pintar nada.

3.2. Objetos que cruzan las fronteras

- **SuperlatticeSpec**: espesores en metros y Drude en rad/s. Lo construye `spec_from_mapping` a partir del YAML.
- **DispersionScan**: vectores ν , R_m , máscara de banda, kL_m/π . Lo produce `scan_dispersion`.
- **FrequencyInterval**: un trozo conexo $[\nu_{\text{mín}}, \nu_{\text{máx}}]$.
- **BandwidthPoint**: anchos y centros a un θ dado (Fig. 6).

4. Capa de configuración

Archivo	Uso
<code>configs/base.yaml</code>	$a = b = 12$ mm, aire, c SI
<code>configs/figure_01.yaml</code>	$\nu_e = \nu_m = 3$ GHz, $m = 3, 4$, cuatro θ
<code>configs/figure_02.yaml</code>	zoom 2–4 GHz, $m = 3 \dots 6$
<code>configs/figure_03.yaml</code>	como Fig. 1 con $\nu_m = 1$ GHz
<code>configs/figure_04.yaml</code>	ventanas densas cerca de 1 GHz
<code>configs/figure_05.yaml</code>	palos vs $m = 2 \dots 8$
<code>configs/figure_06.yaml</code>	$\nu(\theta)$ relleno, $m = 2 \dots 7$ (inferida)

Cada YAML mezcla *física* (espesores, plasmas, polarización) con *numérica de gráfica* (malla de ν , lista de θ , ventanas). `load_spec` / `spec_from_mapping` solo leen la física. El resto lo consume el script de la figura.

Campo `source`: `paper` si el original da los números; `inferred_from_figures_3_to_5` en la Fig. 6.

5. Capa de scripts

`scripts/_common.py` reúne utilidades compartidas: cargar YAML, crear los directorios de datos y de salida de cada figura, guardar archivos `.npz` y JSON, y escribir el sidecar de provenance.

Cada `reproduce_figure_ON.py` hace el mismo esqueleto:

1. `load_figure_config("figure_ON.yaml")`.
2. Construir la malla de ν (y de θ si aplica).
3. Llamar a `scan_dispersion` o a `bandwidth_versus_angle`.
4. Dibujar con `plotting`.
5. Escribir PDF, PNG (300 dpi), SVG, datos y `README.md`.
6. Copiar las imágenes a `figures/reproduced/figure_ON/`.

`reproduce_all.py` lanza los seis scripts en orden, con el mismo intérprete (`sys.executable`), y aborta si uno falla (`check=True`).

`figures/figure_ON/generate.py` es un atajo: ejecuta el script correspondiente con `runpy`.

`analyze_convergence.py` no pinta el paper: estudia cómo $\Delta\nu$ de la Fig. 4 cambia con el número de puntos.

6. Flujo de una corrida

El camino feliz, para *una* figura, es:

```
configs/figure_ON.yaml
|
v
spec_from_mapping      (mm, GHz -> SI)
|
v
SuperlatticeSpec
|
+--> scan_dispersion(spec, m, theta, nu, TE)
|
|           |
|           +--> semitrace_by_recurrence  (por defecto)
|           +--> allowed_mask, arccos
|           v
|           DispersionScan
|
+--> detect_plasmon_modes / allowed_intervals  (Figs. 2, 4, 5)
+--> bandwidth_versus_angle                    (Fig. 6)
|
v
plotting.save_figure
|
+--> figures/figure_ON/output/figure_ON.{pdf,png,svg}
+--> figures/figure_ON/data/*.npz
+--> figures/figure_ON/data/summary.json
+--> figures/figure_ON/data/metadata.json
```

Los PDFs científicos (`paper_reimplementation.pdf`, las tres guías) no recalculan física: incluyen las figuras ya generadas en `figures/figure_ON/output/`. Por eso hay que generar las figuras *antes* de compilar el paper si se cambió el código.

Flujo del Makefile:

```
make all
make tests      -> pytest tests
make figures    -> reproduce_all.py + analyze_convergence.py
make paper      -> pdflatex paper_reimplementation.tex (dos pases)
make docs       -> pdflatex software_documentation.tex
make guides     -> pdflatex guia_conceptual, solucion_numerica,
                  arquitectura; copia los PDF a la raíz
```

Cada objetivo de PDF se copia también a la raíz del repo para abrirlos sin entrar en `docs/`.

7. Tests

`pyproject.toml` configura `testpaths = ["tests"]` y `pythonpath = ["src"]`. No hace falta instalar el paquete para testear.

Archivo	Qué garantiza
<code>test_fibonacci.py</code>	palabras, inflación, N_B , L_m
<code>test_materials.py</code>	ceros de Drude, ν_e , ν_m
<code>test_transfer_matrix.py</code>	$\det T = 1$, eqs. (7)–(9), dos vías de R_m
<code>test_dispersion.py</code>	máscara de banda, gap $\langle n \rangle = 0$
<code>test_validation.py</code>	$N_{\text{modos}} = F_{m-2}$
<code>test_params.py</code>	YAML de las seis figuras

`confptest.py` expone dos fixtures de `SuperlatticeSpec`: plasmas iguales (3, 3 GHz, Figs. 1–2) y partidos (3, 1 GHz, Figs. 3–6).

Tres casos se omiten (`skip`) cuando una identidad de Fibonacci pide $k > m$. El resto debe pasar.

8. Dependencias

8.1. Tiempo de ejecución (Python)

Declaradas en `requirements.txt` y en `pyproject.toml`. Python ≥ 3.10 (`requires-python`). El desarrollo se hizo con Python 3.14.

Paquete	Mínimo	Para qué
<code>numpy</code>	1.24	arrays, $\sqrt{}$, trigonométricas complejas
<code>scipy</code>	1.10	(disponible; el TMM núcleo es NumPy)
<code>matplotlib</code>	3.7	Figs. 1–6
<code>pyyaml</code>	6.0	configs
<code>pytest</code>	7.0	tests (extra <code>dev</code> en <code>pyproject</code>)

No hay CUDA, MPI ni bases de datos. No se descarga nada en tiempo de cálculo: el paper original es un PDF local.

8.2. Sistema, para los PDFs

Además de Python:

- `make`
- `pdflatex` (TeX Live). Estilos usados: `article`, `babel-spanish`, `amsmath`, `graphicx`, `booktabs`, `hyperref`, `listings`, `geometry`, `siunitx`, `enumitem`.

Sin LaTeX se pueden igual ejecutar tests y figuras; fallarán solo `make paper`, `make docs` y `make guides`.

En Arch Linux, un conjunto suficiente es el metapaquete de TeX Live más los estilos anteriores. En Debian/Ubuntu suele bastar `texlive-latex-extra` y `texlive-lang-spanish`.

8.3. Opcional

- Instalar el paquete en modo editable: `pip install -e ".[dev]"` (entonces no hace falta el `sys.path` de los scripts, aunque ellos lo siguen insertando).
- Git: `provenance.py` intenta guardar el hash de HEAD en el JSON de cada figura; si no hay repo, el campo queda vacío.

9. Cómo ejecutar el proyecto

Sustituya `analisis_numerico` por la ruta real del clon.

9.1. 1. Entorno

```
cd analisis_numerico
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Compruebe `python3 --version` ≥ 3.10 . Si `python3` no existe, use el intérprete concreto (por ejemplo `python3.12`).

9.2. 2. Tests (recomendado primero)

```
.venv/bin/pytest
# equivalente:
make tests
```

Debe terminar con todos los casos en verde salvo tres `skipped`.

9.3. 3. Figuras

```
.venv/bin/python scripts/reproduce_all.py
# una sola:
.venv/bin/python scripts/reproduce_figure_04.py
make figures # all + analisis de convergencia
```

Salida esperada: seis bloques “Figura N escrita...” y archivos en `figures/figure_0N/output/`.

9.4. 4. Documentos PDF

```
make paper # paper_reimplementation.pdf
make docs # software_documentation.pdf
make guides # guia_conceptual.pdf, solucion_numerica.pdf,
# arquitectura.pdf
```

`make guides` exige que las figuras existan, porque la guía conceptual incluye los PDF ya generados en `figures/`.

9.5. 5. Todo de una vez

```
source .venv/bin/activate
make all
```

Orden interno: tests → figuras → paper → documentación de software → guías. Si los tests fallan, `make` se detiene y no genera figuras.

9.6. Cambiar un parámetro y volver a correr

Edite el YAML (por ejemplo `omega_m_over_2pi_ghz` en `configs/figure_03.yaml`) y ejecute solo el script de esa figura. No recompila el paper si no necesita el PDF actualizado.

Para TM: ponga `polarization: TM` en el YAML. El original no grafica TM; el código sí la implementa ($\chi = \varepsilon$).

9.7. Limpieza

```
make clean # auxiliares .aux .log .out .toc de docs/
```

No borra figuras ni el entorno virtual.

10. Problemas frecuentes

- `ModuleNotFoundError: fibonacci_tmm`: está ejecutando un script fuera de `scripts/` sin `src` en el path. Use los scripts del repo o `PYTHONPATH=src`.
- `ZeroDivisionError` en $\omega = 0$: el Drude diverge; el YAML no debe incluir el origen en el barrido (la Fig. 1 empieza en 0.15 GHz).
- Fig. 1 sin gaps a $\theta = 0$ y $\omega_e = \omega_m$: no es un bug, es adaptación de impedancia.

- `pdflatex: command not found`: instale TeX Live; tests y figuras no lo necesitan.
- Figuras viejas en el paper: regenerar con `reproduce_all.py` y luego `make paper`.
- El Makefile apunta a `.venv/bin/python`. Si el venv está en otra ruta, invoque los comandos con el intérprete explícito.

11. Dónde está cada documento

PDF en la raíz	Contenido
<code>paper_reimplementation.pdf</code>	artículo de la reproducción
<code>software_documentation.pdf</code>	manual corto de la API
<code>guia_conceptual.pdf</code>	física desde cero
<code>solucion_numerica.pdf</code>	TMM y código de las figuras
<code>arquitectura.pdf</code>	este documento

Fuentes LaTeX en `docs/`. Auditoría previa al código: `docs/fase_A_auditoria_matematica.md`. Informe de validación: `docs/INFORME_VALIDACION.md`. Paper original: PDF en la raíz del repo.

12. Resumen operativo

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
pytest
python scripts/reproduce_all.py
make paper docs guides
```

Eso deja tests en verde, las seis figuras y todos los PDF en la raíz del proyecto.